智能数字人 RAG 问答系统

系统使用说明书

软件名称	智能数字人 RAG 问答系统
软件版本	V2.0.0
开发语言	Python
运行环境	Windows/Linux/Mac
文档版本	V1.0
编写日期	2025 年 10 月 29 日
适用对象	系统管理员、开发人员、最终用户
文档类型	系统使用说明书

目录

1	系统概述	3
1.1	系统简介	3
1.2	系统特色	3
1.3	技术架构	4
1.4	功能模块	5
2	系统安装	6
2.1	环境要求	6
2.2	安装步骤	6
2.3	启动验证	7
3	系统配置	8
3.1	基础配置	8
3.2	模型配置	9
3.3	语音配置	10
4	功能使用说明	11
4.1	Web 界面使用	11
4.2	文档管理	13
4.3	问答功能	15
4.4	语音交互	17
4.5	数字人控制	19
5	API 接口说明	21
5.1	HTTP API	21
5.2	WebSocket API	23
5.3	接口示例	25
6	数字人对接说明	27
6.1	UE5 对接	27

6.2	Unity 对接	29
6.3	消息协议	31
7	故障排除	33
7.1	常见问题	33
7.2	错误代码	35
7.3	日志分析	37
8	附录	39
8.1	配置文件说明	39
8.2	技术参数	41
8.3	更新历史	43

1. 系统概述

1.1 系统简介

智能数字人 RAG 问答系统是一个集成了检索增强生成（RAG）技术、语音交互、情感分析、数字人控制等多项人工智能技术的综合性智能问答系统。

系统基于 Python 开发，采用模块化架构设计，支持多种文档格式的知识库构建，能够提供准确的智能问答服务，并具备完整的数字人交互能力。

主要应用场景包括：

- 智能客服数字人
- 虚拟教师/讲师
- 展厅数字导览
- 企业知识助手
- 医疗咨询助手

1.2 系统特色

本系统具有以下核心特色：

1. 专业 RAG 问答
    - 支持 PDF、Word、TXT、Markdown 等多种文档格式
    - 基于 Chroma 向量数据库的智能检索
    - 集成 Qwen 大语言模型
    - 智能对话记忆管理
    - 智能回退机制

2. 完整语音交互

- FunASR 离线语音识别，完全免费
- Edge TTS 高质量语音合成
- 支持 4 种声音角色和 6 种情感风格
- 语速可调节（0.5-2.0 倍）
- 智能音频缓存

3. 数字人控制
   - 8 种情感自动识别
   - 40+ Blend Shapes 表情控制
   - 15 种 Viseme 唇形同步
   - UE5/Unity 标准格式导出
   - 实时 WebSocket 通信

4. 技术优势
   - 模块化架构，易于扩展
   - 异步处理，支持高并发
   - 完全免费方案，无 API 费用
   - 本地部署，数据安全
   - 标准化接口，易于集成

1.3 技术架构

系统采用分层架构设计，主要包含以下层次：

1. 用户交互层
   - Web UI 界面
   - UE5 数字人引擎
   - Unity 数字人引擎
   - 移动端应用

2. 通信层
    - WebSocket 实时通信（端口 10003）
    - HTTP RESTful API（端口 5000）
    - 消息队列管理
    - 多用户并发支持

3. 智能处理层
    - ASR 语音识别（FunASR）
    - RAG 检索（Chroma 向量数据库）
    - LLM 生成（Qwen 大模型）
    - 情感分析（关键词匹配）
    - 表情控制（Blend Shapes）
    - TTS 语音合成（Edge TTS）
    - 唇形同步（Viseme 生成）

4. 数据层
    - 向量数据库（Chroma）
    - 文档存储（本地文件系统）
    - 音频缓存（MP3 文件）
    - 会话数据（SQLite）
    - 对话记忆（向量存储）

技术栈：

- 后端框架：Flask + AsyncIO
- AI 模型：Qwen + ChromaDB + FunASR + Edge TTS
- 数据库：SQLite + Chroma

- 通信协议：WebSocket + HTTP
- 音频处理：Librosa + Pydub

2. 系统安装

2.1 环境要求

硬件要求：

- CPU: Intel i5 或 AMD 同等性能处理器
- 内存: 8GB RAM（推荐 16GB）
- 存储: 20GB 可用磁盘空间
- 网络: 稳定的互联网连接（用于模型下载）

软件要求：

- 操作系统: Windows 10/11, Linux Ubuntu 18.04+, macOS 10.15+
- Python: 3.8 或更高版本
- pip: 最新版本
- Git: 用于代码管理（可选）

可选要求（性能优化）：

- NVIDIA GPU: 支持 CUDA 11.7+
- VRAM: 4GB+（使用 GPU 加速时）

2.2 安装步骤

步骤 1：获取源代码

1. 解压系统安装包到目标目录
2. 或使用 Git 克隆代码仓库：

```
git clone <repository_url>
cd intelligent_qa_system
```

步骤 2：创建 Python 虚拟环境（推荐）

```
# Windows
python -m venv venv
venv\Scripts\activate

# Linux/Mac
python3 -m venv venv
source venv/bin/activate
```

步骤 3：安装依赖包

```
pip install -r requirements.txt
```

注意：首次安装可能需要较长时间，因为需要下载 AI 模型文件。

步骤 4：配置系统

1. 复制 config/config.yaml.example 为 config/config.yaml
2. 根据需要修改配置参数（详见第 3 章）

步骤 5：初始化数据目录

系统会在首次启动时自动创建必要的数据目录：

- data/documents - 文档存储
- data/vectordb - 向量数据库
- data/cache - 缓存文件
- logs - 日志文件

2.3 启动验证

启动系统：

```
python main.py
```

预期输出：

```
============================================================
智能问答系统启动中...
============================================================
✓ 配置加载成功: QA System
✓ 数据目录创建完成
✓ Embedding 模型加载成功
✓ 问答系统初始化完成!
============================================================
☑ 数字人 WebSocket 服务器已在后台启动
   监听地址: ws://0.0.0.0:10003
   兼容 Fay 协议，支持流式 RAG 问答
============================================================
服务地址: http://0.0.0.0:5000
系统启动成功，准备接收请求...
============================================================
```

验证步骤：

1. 打开浏览器访问 http://localhost:5000
2. 查看系统主界面是否正常显示
3. 尝试上传一个测试文档
4. 进行简单的问答测试

如果出现错误，请参考第 7 章故障排除部分。

3. 系统配置

3.1 基础配置

主配置文件位置：config/config.yaml

基础配置项：

```yaml
# 应用基础配置
app_name: "智能问答系统"
version: "2.0.0"
host: "0.0.0.0"
port: 5000
debug: false

# 日志配置
logging:
  level: "INFO"
  file: "logs/qa_system.log"
  max_size: "10MB"
  backup_count: 5
```

配置说明：

- app_name: 应用名称，显示在界面标题
- version: 系统版本号
- host: 服务监听地址，0.0.0.0 表示监听所有网卡
- port: HTTP 服务端口，默认 5000

- debug: 调试模式，生产环境建议设为 false
- logging: 日志相关配置

3.2 模型配置

AI 模型配置：

```yaml
# LLM 配置
llm:
  provider: "qwen"
  model_name: "qwen-plus"
  api_key: "your_api_key_here"
  base_url: "https://dashscope.aliyuncs.com/compatible-mode/v1"
  temperature: 0.7
  max_tokens: 2000

# Embedding 模型配置
embedding:
  model_name: "BAAI/bge-large-zh-v1.5"
  device: "cpu"  # 或 "cuda" 如果有 GPU
  batch_size: 32

# 向量数据库配置
vector_db:
  provider: "chroma"
  persist_directory: "data/vectordb"
  collection_name: "documents"
  distance_metric: "cosine"
```

配置说明：

- provider: LLM 提供商，支持 qwen、openai 等
- model_name: 具体的模型名称
- api_key: API 密钥，从服务商获取
- temperature: 生成温度，控制回答的随机性
- device: 计算设备，有 GPU 时可设为 cuda
- batch_size: 批处理大小，影响处理速度

3.3 语音配置

语音功能配置：

```yaml
# 数字人集成配置
fay_integration:
  enabled: true
  websocket:
    host: "0.0.0.0"
    port: 10003
    enable_voice_output: true

# 语音配置
voice:
  asr:
    engine: "funasr"  # 语音识别引擎
    model: "paraformer-zh"
    device: "cpu"

  tts:
```

```
    engine: "edge"  # 语音合成引擎
    default_voice: "female"
    default_emotion: "friendly"
    cache_enabled: true
    cache_dir: "data/audio_cache"
```

配置说明：

- enabled: 是否启用数字人功能
- websocket: WebSocket 服务器配置
- asr: 语音识别相关配置
- tts: 语音合成相关配置
- engine: 引擎选择，funasr 免费，edge 免费
- cache_enabled: 是否启用音频缓存

4. 功能使用说明

4.1 Web 界面使用

4.1.1 访问系统

启动系统后，打开浏览器访问：http://localhost:5000

4.1.2 主界面介绍

系统主界面包含以下区域：

- 顶部导航栏：系统名称和功能菜单
- 左侧边栏：文档管理、会话历史
- 中央区域：问答对话界面
- 右侧面板：系统状态、相关推荐

4.1.3 基本操作

1. 文档上传
   - 点击"上传文档"按钮
   - 选择 PDF、Word、TXT 或 Markdown 文件
   - 等待文档处理完成
   - 查看处理状态和结果

2. 问答交互
   - 在输入框中输入问题
   - 点击"发送"按钮或按 Enter 键
   - 观察系统实时生成答案
   - 查看答案来源和相关推荐

3. 会话管理

- 系统自动保存对话历史
- 可在左侧边栏查看历史会话
- 点击历史会话可继续对话
- 支持删除不需要的会话

4.2 文档管理

4.2.1 支持的文档格式

- PDF 文件（.pdf）
- Word 文档（.docx, .doc）
- 文本文件（.txt）
- Markdown 文件（.md）
- Excel 文件（.xlsx, .xls）

4.2.2 文档上传流程

1. 准备文档
   - 确保文档内容清晰、结构良好
   - 文件大小建议不超过 50MB
   - 文档语言主要支持中文和英文

2. 上传操作
   - 点击"文档管理" → "上传文档"
   - 拖拽文件到上传区域或点击选择文件
   - 填写文档描述信息（可选）
   - 点击"开始上传"

3. 处理过程
   - 文档解析：提取文本内容
   - 文本分割：切分为适当的片段

- 向量化：生成文档向量表示
- 索引构建：添加到向量数据库

4.2.3 文档管理操作

- 查看文档列表：显示所有已上传的文档
- 文档详情：查看文档内容和处理状态
- 删除文档：从系统中移除文档及其索引
- 重新处理：重新解析和索引文档
- 批量操作：支持批量上传和删除

4.3 问答功能

4.3.1 问答模式

系统支持多种问答模式：

1. 文档问答模式
   - 基于已上传文档回答问题
   - 提供答案来源和相关片段
   - 支持多文档综合回答

2. 通用问答模式
   - 当文档信息不足时自动启用
   - 基于大模型的通用知识回答
   - 智能回退机制保证有答案

3. 混合问答模式
   - 结合文档知识和通用知识
   - 提供更全面的答案
   - 标注信息来源

4.3.2 提问技巧

为获得更好的回答效果，建议：

- 问题表达清晰具体
- 避免过于宽泛的问题
- 可以包含上下文信息
- 使用关键词提高检索精度

示例：

好的问题："产品 X 的技术规格是什么？"

不好的问题："告诉我一些信息"

4.3.3 答案解读

系统回答包含以下信息：

- 主要答案：直接回答用户问题
- 来源文档：答案来源的文档名称
- 相关片段：支持答案的原文片段
- 置信度：系统对答案的信心程度
- 相关问题：推荐的相关问题

5. API 接口说明

5.1 HTTP API

5.1.1 基础信息

- 基础 URL: http://localhost:5000/api
- 内容类型: application/json
- 字符编码: UTF-8
- 认证方式: 暂不需要（可根据需要添加）

5.1.2 主要接口

1. 健康检查

GET /health

返回系统运行状态

2. 文档上传

POST /upload

Content-Type: multipart/form-data

参数：

- file: 文档文件
- description: 文档描述（可选）

3. 问答接口

POST /question

参数：

```
{
  "question": "用户问题",
```

```
  "session_id": "会话 ID（可选）",
  "mode": "问答模式（可选）"
}
```

4. 流式问答

POST /question/stream

参数同上，返回流式数据

5. 文档列表

GET /documents

返回已上传的文档列表

6. 删除文档

DELETE /documents/{document_id}

删除指定文档

5.1.3 响应格式

成功响应：

```
{
  "success": true,
  "data": {...},
  "message": "操作成功"
}
```

错误响应：

```
{
  "success": false,
  "error": "错误信息",
```

```
  "code": "错误代码"
}
```

5.2 WebSocket API

5.2.1 连接信息

- WebSocket URL: ws://localhost:10003
- 协议: WebSocket
- 消息格式: JSON

5.2.2 消息类型

客户端发送消息：

1. 文本输入

```
{
  "type": "text_input",
  "data": {
    "text": "问题文本",
    "user_id": "用户 ID",
    "session_id": "会话 ID"
  }
}
```

2. 语音输入

```
{
  "type": "voice_input",
  "data": {
    "audio": "Base64 音频数据",
    "format": "音频格式（wav/mp3）",
```

```
    "user_id": "用户 ID",
    "session_id": "会话 ID"
  }
}
```

3. 打断指令

```
{
  "type": "interrupt",
  "data": {}
}
```

服务器发送消息：

1. 欢迎消息

```
{
  "type": "welcome",
  "data": {
    "server": "服务器信息",
    "version": "版本号",
    "client_id": "客户端 ID"
  }
}
```

2. 思考状态

```
{
  "type": "thinking",
  "data": {
    "status": "当前状态",
    "message": "状态描述"
```

```
  }
}
```

3. 流式内容

```
{
  "type": "content",
  "data": "文本片段"
}
```

4. 表情控制

```
{
  "type": "expression",
  "data": {
    "emotion": "情感类型",
    "blend_shapes": {...}
  }
}
```

5. 语音输出

```
{
  "type": "voice_output",
  "data": {
    "audio_url": "音频文件 URL",
    "text": "原始文本",
    "lipsync": {...}
  }
}
```

6. 数字人对接说明

6.1 UE5 对接

6.1.1 环境准备

- Unreal Engine 5.0+
- BlueprintWebsockets 插件
- MetaHuman 角色资源

6.1.2 WebSocket 连接

在 UE5 Blueprint 中：

1. 创建 WebSocket 连接
   - 添加 WebSocket 组件
   - 设置连接 URL: ws://localhost:10003
   - 绑定连接事件

2. 消息处理
   - 绑定 OnMessage 事件
   - 解析 JSON 消息
   - 根据消息类型执行相应操作

6.1.3 表情控制实现

接收 expression 消息时：

1. 解析 blend_shapes 数据
2. 遍历所有 Blend Shape 权重
3. 应用到 MetaHuman 角色的面部网格
4. 设置过渡时间实现平滑变化

Blueprint 节点示例：

- Get Skeletal Mesh Component
- Set Morph Target
- 循环设置所有 Blend Shape 权重

6.1.4 语音播放和唇形同步

接收 voice_output 消息时：

1. 下载音频文件
2. 创建 Sound Wave 资源
3. 播放音频
4. 同时应用 lipsync 数据到口型 Blend Shapes
5. 按时间轴同步播放

6.1.5 完整示例代码

```cpp
// C++代码示例
void ADigitalHumanCharacter::OnWebSocketMessage(const FString& Message)
{
    TSharedPtr<FJsonObject> JsonObject;
    TSharedRef<TJsonReader<>> Reader =
TJsonReaderFactory<>::Create(Message);

    if (FJsonSerializer::Deserialize(Reader, JsonObject))
    {
        FString MessageType = JsonObject->GetStringField("type");

        if (MessageType == "expression")
```

```
        {
            HandleExpressionMessage(JsonObject->GetObjectField("data"));
        }
        else if (MessageType == "voice_output")
        {
            HandleVoiceOutputMessage(JsonObject->GetObjectField("data"));
        }
    }
}
```

6.2 Unity 对接

6.2.1 环境准备

- Unity 2021.3 LTS+
- WebSocket 库（如 WebSocketSharp）
- 数字人角色模型

6.2.2 WebSocket 客户端实现

```csharp
using WebSocketSharp;
using UnityEngine;
using Newtonsoft.Json.Linq;

public class DigitalHumanController : MonoBehaviour
{
    private WebSocket ws;
    private SkinnedMeshRenderer faceRenderer;
```

```
    void Start()
    {
        // 初始化 WebSocket 连接
        ws = new WebSocket("ws://localhost:10003");
        ws.OnMessage += OnWebSocketMessage;
        ws.Connect();

        // 获取面部渲染器
        faceRenderer = GetComponent<SkinnedMeshRenderer>();
    }

    void OnWebSocketMessage(object sender, MessageEventArgs e)
    {
        var json = JObject.Parse(e.Data);
        string messageType = json["type"].ToString();

        switch (messageType)
        {
            case "expression":
                HandleExpression(json["data"]);
                break;
            case "voice_output":
                HandleVoiceOutput(json["data"]);
                break;
        }
    }
}
```

6.2.3 Blend Shapes 控制

```csharp
void HandleExpression(JToken data)
{
    var blendShapes = data["blend_shapes"];

    foreach (var shape in blendShapes)
    {
        string shapeName = shape.Path;
        float weight = (float)shape.Value * 100f; // 转换为 0-100 范围

        int shapeIndex =
faceRenderer.sharedMesh.GetBlendShapeIndex(shapeName);
        if (shapeIndex >= 0)
        {
            faceRenderer.SetBlendShapeWeight(shapeIndex, weight);
        }
    }
}
```

6.2.4 音频播放和唇形同步

```csharp
void HandleVoiceOutput(JToken data)
{
    string audioUrl = data["audio_url"].ToString();
    var lipsyncData = data["lipsync"];
```

```
    // 下载并播放音频
    StartCoroutine(PlayAudioFromURL(audioUrl));

    // 应用唇形数据
    StartCoroutine(ApplyLipsyncData(lipsyncData));
}

IEnumerator ApplyLipsyncData(JToken lipsyncData)
{
    var visemes = lipsyncData["visemes"];

    foreach (var viseme in visemes)
    {
        float timestamp = (float)viseme["timestamp"];
        string visemeName = viseme["viseme"].ToString();
        float weight = (float)viseme["weight"];

        // 等待到指定时间
        yield return new WaitForSeconds(timestamp);

        // 应用口型权重
        ApplyVisemeWeight(visemeName, weight);
    }
}
```

7. 故障排除

7.1 常见问题

7.1.1 安装和启动问题

Q: 安装依赖时出现网络错误?

A:

1. 检查网络连接是否正常
2. 配置 pip 镜像源:

   pip config set global.index-url

https://pypi.tuna.tsinghua.edu.cn/simple

3. 使用代理: pip install --proxy http://proxy:port package_name

Q: 启动时提示"模块未找到"错误?

A:

1. 确认已激活虚拟环境
2. 重新安装依赖: pip install -r requirements.txt
3. 检查 Python 版本是否为 3.8+

Q: 系统启动后无法访问 Web 界面?

A:

1. 检查防火墙设置, 确保 5000 端口未被阻止
2. 尝试使用 127.0.0.1:5000 而不是 localhost:5000
3. 查看启动日志中是否有错误信息

7.1.2 功能使用问题

Q: 上传文档后无法检索到相关内容?

A:

1. 检查文档格式是否支持
2. 确认文档内容不为空
3. 等待文档处理完成（大文档需要较长时间）
4. 尝试重新处理文档

Q: 语音功能无法正常工作？

A:

1. 检查麦克风权限设置
2. 确认浏览器支持 WebRTC
3. 查看 Edge TTS 服务是否可用
4. 尝试更新 edge-tts 包：pip install edge-tts --upgrade

Q: WebSocket 连接失败？

A:

1. 确认 10003 端口未被占用
2. 检查防火墙设置
3. 查看系统日志中的 WebSocket 相关错误
4. 尝试重启系统

7.1.3 性能问题

Q: 系统响应速度慢？

A:

1. 检查系统资源使用情况
2. 减少并发用户数量
3. 优化向量数据库大小
4. 考虑使用 GPU 加速

Q: 内存占用过高?

A:

1. 关闭不必要的应用程序
2. 调整 batch_size 参数
3. 使用模型量化
4. 定期清理缓存文件

7.2 错误代码

系统错误代码说明:

E001: 配置文件加载失败
- 原因：config.yaml 文件不存在或格式错误
- 解决：检查配置文件路径和格式

E002: 模型加载失败
- 原因：AI 模型文件损坏或网络问题
- 解决：重新下载模型或检查网络连接

E003: 数据库连接失败
- 原因：数据库文件权限问题或磁盘空间不足
- 解决：检查文件权限和磁盘空间

E004: API 密钥无效
- 原因：LLM API 密钥错误或过期
- 解决：更新 config.yaml 中的 API 密钥

E005: 文档处理失败

- 原因：文档格式不支持或文件损坏
- 解决：检查文档格式和完整性

E006: WebSocket 连接异常
- 原因：端口被占用或网络问题
- 解决：检查端口状态和网络连接

E007: 语音服务不可用
- 原因：TTS/ASR 服务异常
- 解决：检查网络连接或切换服务提供商

E008: 内存不足
- 原因：系统内存不够或内存泄漏
- 解决：增加内存或重启系统

E009: 权限不足
- 原因：文件或目录访问权限问题
- 解决：调整文件权限或以管理员身份运行

E010: 未知错误
- 原因：系统异常或代码 bug
- 解决：查看详细日志或联系技术支持

8. 附录

8.1 配置文件说明

完整的 config.yaml 配置文件示例：

```yaml
# 应用基础配置
app_name: "智能数字人 RAG 问答系统"
version: "2.0.0"
host: "0.0.0.0"
port: 5000
debug: false

# 日志配置
logging:
  level: "INFO"
  file: "logs/qa_system.log"
  max_size: "10MB"
  backup_count: 5
  format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"

# LLM 配置
llm:
  provider: "qwen"
  model_name: "qwen-plus"
  api_key: "your_api_key_here"
  base_url: "https://dashscope.aliyuncs.com/compatible-mode/v1"
```

```
  temperature: 0.7
  max_tokens: 2000
  timeout: 30

# Embedding 模型配置
embedding:
  model_name: "BAAI/bge-large-zh-v1.5"
  device: "cpu"
  batch_size: 32
  max_length: 512

# 向量数据库配置
vector_db:
  provider: "chroma"
  persist_directory: "data/vectordb"
  collection_name: "documents"
  distance_metric: "cosine"
  n_results: 5

# 文档处理配置
document_processing:
  chunk_size: 1000
  chunk_overlap: 200
  supported_formats: ["pdf", "docx", "txt", "md"]
  max_file_size: "50MB"

# 数字人集成配置
fay_integration:
```

```
  enabled: true
  websocket:
    host: "0.0.0.0"
    port: 10003
    enable_voice_output: true
    max_connections: 100

# 语音配置
voice:
  asr:
    engine: "funasr"
    model: "paraformer-zh"
    device: "cpu"
    sample_rate: 16000

  tts:
    engine: "edge"
    default_voice: "female"
    default_emotion: "friendly"
    cache_enabled: true
    cache_dir: "data/audio_cache"
    max_cache_size: "1GB"

# 情感分析配置
emotion:
  enabled: true
  confidence_threshold: 0.6
  supported_emotions: ["happy", "sad", "angry", "surprised", "fearful",
```

```
"curious", "thinking", "neutral"]

# 会话管理配置
session:
  max_history_length: 10
  session_timeout: 3600
  memory_enabled: true
```

8.2 技术参数

系统技术参数：

性能参数：

- 并发用户数：最大 100 个 WebSocket 连接
- 响应时间：文本问答 2-5 秒，语音问答 5-8 秒
- 吞吐量：每秒处理 10-20 个请求
- 内存占用：2-4GB（包含 AI 模型）
- 磁盘占用：基础安装 2GB，运行时按需增长

支持格式：

- 文档格式：PDF, DOCX, DOC, TXT, MD, XLSX, XLS
- 音频格式：WAV, MP3, OGG
- 图片格式：JPG, PNG, GIF（文档中的图片）
- 视频格式：暂不支持

网络协议：

- HTTP/1.1：Web 界面和 API 接口
- WebSocket：实时通信

- TCP：数据库连接
- HTTPS：可配置 SSL 支持

数据库：

- SQLite：会话和用户数据
- ChromaDB：向量数据存储
- 文件系统：文档和音频文件

AI 模型：

- LLM：Qwen 系列模型
- Embedding：BGE 中文向量模型
- ASR：FunASR 达摩院模型
- TTS：Microsoft Edge TTS

安全特性：

- 本地部署：数据不出本地
- 文件验证：上传文件格式和大小检查
- 输入过滤：防止恶意输入
- 访问控制：可配置 IP 白名单

8.3 更新历史

版本更新历史：

V2.0.0 (2025-10-29)

- 新增数字人集成功能
- 新增 WebSocket 实时通信
- 新增语音识别和语音合成
- 新增情感分析和表情控制

- 新增唇形同步功能
- 优化 RAG 检索算法
- 改进用户界面设计
- 增强系统稳定性

V1.5.0 (2025-09-15)
- 新增流式问答功能
- 新增会话记忆管理
- 新增智能回退机制
- 优化文档处理性能
- 修复已知 bug

V1.0.0 (2025-08-01)
- 基础 RAG 问答功能
- 多格式文档支持
- 向量数据库集成
- Web 界面实现
- API 接口提供

开发路线图：
- V2.1.0：增强移动端支持
- V2.2.0：多语言支持
- V2.3.0：高级情感模型
- V3.0.0：分布式部署支持

技术支持：
- 文档：查看项目文档目录
- 日志：logs/qa_system.log

- 社区：GitHub Issues
- 邮箱：support@example.com